CENG-567
REINFORCEMENT LEARNING

Lecture-7

Monte Carlo Prediction

Dr. Osman GÖKALP

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap (**From MRP → MDP & Policies**)

Markov Decision Process (MDP)

MDP = MRP **+ actions**

Decisions affect **transitions** P(s'|s,a) and **rewards** R(s,a)

Formal: **⟨S, A, P, R, γ⟩**

Foundations of decision-making under uncertainty

Goal:
Find a **policy** π that maximizes expected cumulative reward.

Policy (π):

Maps states → actions (deterministic or stochastic)

Under a policy, the MDP reduces to an **MRP** (when we fix a policy).

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap (**Value Functions & Bellman Equations)**

State-value under policy π:

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma\, v_\pi(s')]$$

Action-value under policy π:

$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\,[\mathrm{r} + \gamma \mathrm{v}_\pi(\mathrm{s}')]$$

Bellman Expectation (Policy Evaluation):

Recursive relation of values under policy π

Basis for iterative policy evaluation

Optimal Value Functions:

$$\mathrm{v}_*(\mathrm{s}) = \max_a \mathrm{q}_*(\mathrm{s}, a)$$

Bellman Optimality Equation gives recursive solution for optimal values

Unique v_*; several policies may achieve it

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap (**Solving RL & DP Drawbacks)**

Solving RL: Three Major Approaches

Dynamic Programming (DP): requires full knowledge of environment model

Monte Carlo: model-free, learns from complete episodes

Temporal Difference (TD): model-free, updates incrementally at each step

DP Recap:

Policy Evaluation – Find *v* values for a given policy until convergence

Policy Improvement – First perform policy evaluation and obtain *v*. Then, based on *v*, find best action for each state, update policy accordingly. Repeat until no change in the policy.

Value Iteration – Start with zero *v* values (no policy evaluation). Based on current v, find best action for each state, update policy accordingly. Repeat until *v* values converge.

Drawbacks of DP:

Curse of Dimensionality: large state spaces → huge tables, slow computation

Requires Complete Model: transition probabilities often unknown in practice

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Why Monte Carlo Methods?

Training an agent with Monte Carlo (MC)

Suppose we want to estimate the probability of a biased coin landing on heads.

- **Analytical approach**: Model the physics of the coin → exact but impractical.
- **Sampling approach**: Flip the coin many times → simpler; approximate but useful.

MC methods follow the **sampling approach**: learn from repeated trials instead of analyzing full dynamics.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

MC in Reinforcement Learning

Key idea: Estimate values from **sample episodes**, not from known environment models.

MC is useful because real environments:

Are **too complex** to model analytically, or

Have **unknown transition probabilities** and reward distributions.

MC uses:

Trajectories of complete episodes (from start to terminal) generated by interacting with the environment

Averaged returns to estimate:

State values $v_\pi(s)$
Action values $\mathrm{Q}_\pi(s, a)$

MC gives a practical way to learn **only from experience**, without knowing the environment's internal mechanics.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Monte Carlo Prediction

Remember: how to improve a policy?

- Need to evaluate a given policy
- Evaluation process needs: state and action values

Remember: the value of a state s under policy π:

$$v_\pi(s) = E_\pi[\, G_t \mid S_t = s \,] = E_\pi[\, \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \,]$$

MC prediction can **estimate state value** by **observing** sample **trajectories**.

Trajectory is a sequence of **'state-action-reward'** (sar) tuples.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Using Sample Trajectories for State-Value Estimation

Example: Grid-World Robot

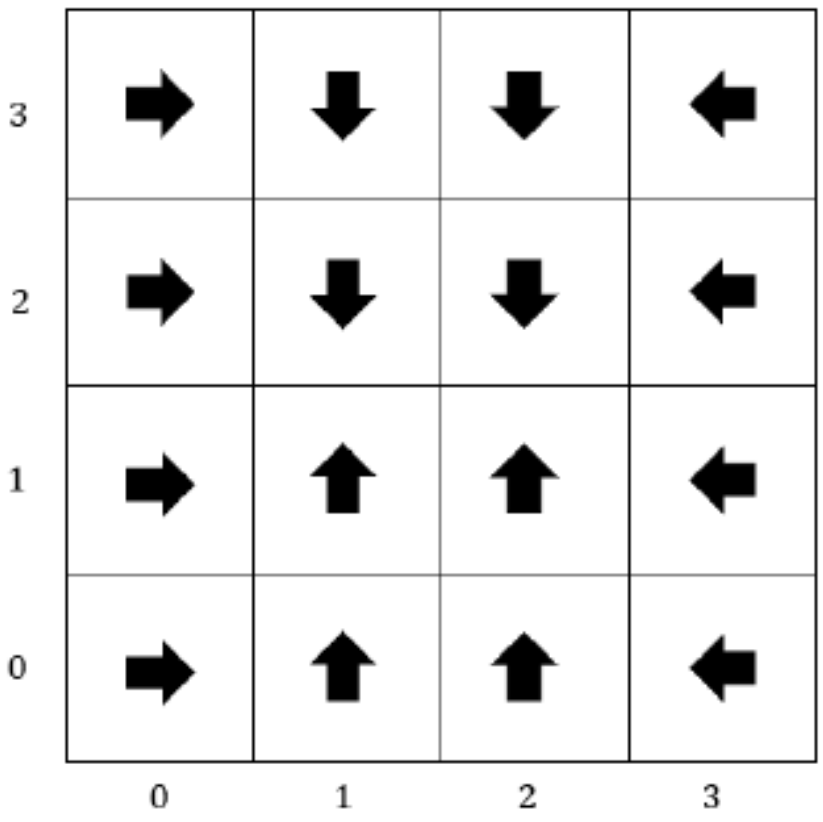
Robot receives **+1 reward** for every move that does **not** crash.

If it **hits a wall**, the robot **crashes → episode ends**.

Robot follows commands **imperfectly**:

- 70% → moves in commanded direction
- 10% → moves in each of the other three directions

Given: A **deterministic policy** π



Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example Trajectories Under Policy π

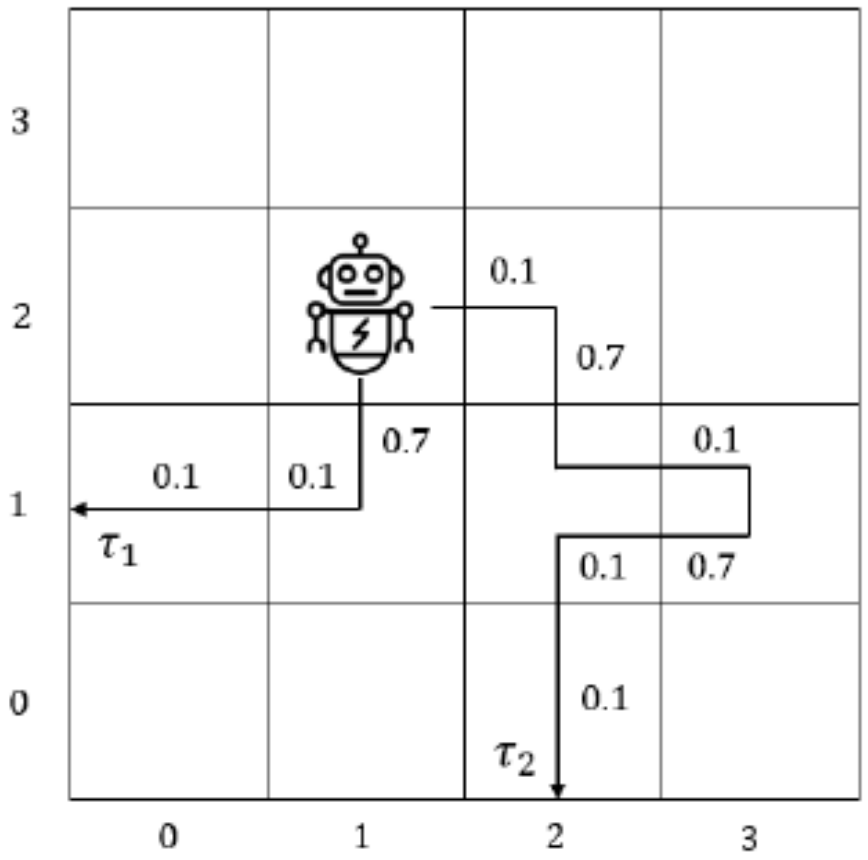
For starting state **(1,2)**:

Due to action stochasticity, the robot may follow different trajectories even under a deterministic policy.

Two example trajectories: τ_1, τ_2

Each trajectory has its own **sequence of transition probabilities**.

*Although the **policy is deterministic** and always issues the same action in a given state, the robot's trajectory is random because the **environment introduces stochastic transitions**.*



Discounted Returns for the Example Trajectories τ_1, τ_2

$$\tau_1: S_0 = (1,2), A_0 = down,$$
$$R_1 = 1, S_1 = (1,1), A_1 = up,$$
$$R_2 = 1, S_2 = (0,1), A_2 = right,$$
$$R_3 = 0, S_3 = crashed$$
$$p_\pi(\tau_1) = 0.7 \cdot 0.1 \cdot 0.1 = 0.007$$
$$G_{\tau_1} = 1 + \gamma 1 + \gamma^2 0$$

$$\tau_2: S_0 = (1,2), A_0 = down,$$
$$R_1 = 1, S_1 = (2,2), A_1 = down,$$
$$R_2 = 1, S_2 = (2,1), A_2 = up,$$
$$R_3 = 1, S_3 = (3,1), A_3 = left,$$
$$R_4 = 1, S_4 = (2,1), A_4 = up,$$
$$R_5 = 1, S_5 = (2,0), A_5 = up,$$
$$R_6 = 0, S_6 = crashed$$
$$p_\pi(\tau_2) = 0.1 \cdot 0.7 \cdot 0.1 \cdot 0.7 \cdot 0.1 \cdot 0.1 = 0.000049$$
$$G_{\tau_2} = 1 + \gamma 1 + \gamma^2 1 + \gamma^3 1 + \gamma^4 1 + \gamma^5 0$$

State Value from the Discounted Returns

$$v_\pi(s) = E_\pi[\, G_t \mid S_t = s \,] = E_\pi[\, \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \,] = \sum_{\mathrm{i}} \mathrm{p}_\pi(\tau_\pi).\, \mathrm{G}_{\tau_\mathrm{i}}$$

To calculate true state values we need to identify:

Every single possible trajectory τ

The **probability** of observing τ under π

The corresponding **discounted return** G_t

However, in practice this does not work. Even in this simple problem, there is an infinite number of possible trajectories!

Solution: Monte Carlo prediction

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Monte Carlo Prediction

$$\hat{v}_\pi(s) = \frac{G_{\tau_1} + \cdots + G_{\tau_\mathrm{k}}}{\mathrm{k}}$$

We can calculate the **estimate** of state value by averaging the sample returns as above.

In our 2-sample trajectory example:

$$\hat{v}_\pi(1,2) = \frac{G_{\tau_1} + G_{\tau_2}}{2}$$

- Note that v_π denotes the true value of the state, whereas $\hat{v}_\pi$ denotes an estimate.

Monte Carlo Prediction

Common questions:

1) "Is using only two sample returns enough to estimate a value based on infinitely many possible trajectories?"
→ No. More trajectories improve the accuracy of the estimate.

2) "How do we know when we have enough samples?"
→ Hard to measure precisely.
→ More complex or more stochastic environments require more samples.
→ A practical approach: check whether value estimates **converge** as more trajectories are added.

3) "Trajectories τ_1 and τ_2 have very different probabilities. Is it valid to weight them equally?"
→ With only two samples, it is problematic.
→ But as the sample size grows, trajectories appear in proportion to their true probabilities.

4) "Can one trajectory be used to estimate values of every state it visits?"
→ **Yes.** Monte Carlo prediction uses each episode to update the value of all states encountered in that episode.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

First-visit vs. Every-visit Monte Carlo Prediction

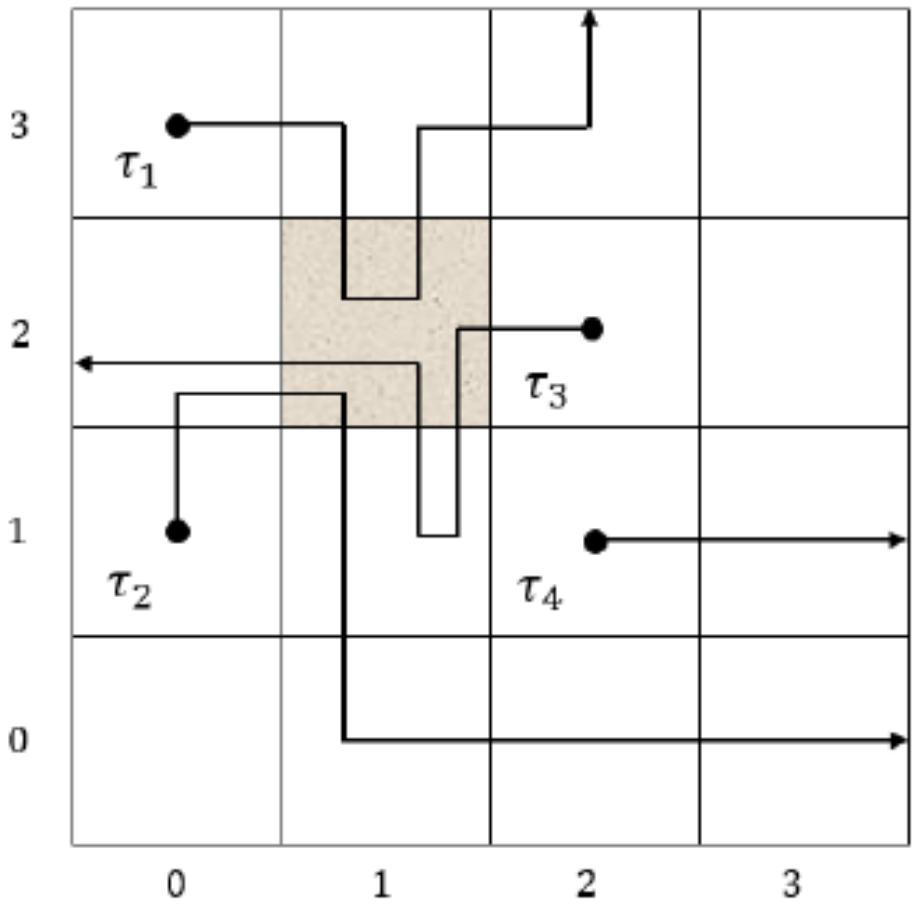
- Markov property: future depends on the current state, not the past.
- So, for example, we can think of $\boldsymbol{\tau_1}$ as **three separate trajectories** that originate from states (1,2), (1,1), and (0,1). Let's call the latter two trajectories $\boldsymbol{\tau_1'}$ and $\boldsymbol{\tau_1''}$.
- Since there is no other trajectory visiting state (1,1), we used a single return to estimate its state-value:

$$\hat{v}_\pi(1,1) = \frac{\mathrm{G}_{\tau_1'}}{1}$$

$$\mathrm{G}_{\tau_1'} = 1 + \gamma 0$$

First-visit vs. Every-visit Monte Carlo Prediction

- Given the figure here, how can we calculate the state value estimate for the state (1,2)?
- None of the sample trajectories actually originate from state (1,2) but that is totally fine. We can use trajectories $\boldsymbol{\tau_1}$, $\boldsymbol{\tau_2}$, and $\boldsymbol{\tau_3}$ for the estimation.
- **The problem: $\boldsymbol{\tau_3}$ visits (1,2) twice!**
- *Should we use the return only from its* ***first visit*** *or from* ***each of its visits****?*



Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

First-visit vs. Every-visit Monte Carlo Prediction

<u>Both approaches are valid:</u>

First-Visit MC: Use only the first time a state appears in an episode

Every-Visit MC: Use *all* occurrences of a state in an episode

Comparison:

Both converge to the true value $v_\pi(s)$ as the number of visits → ∞

First-Visit MC:

- Provides an **unbiased** estimate
- Has **higher MSE** with small samples
- But achieves **lower MSE** than every-visit MC with many samples

Every-Visit MC:

- Estimates are **biased** (these multiple returns **are not independent**) (bias **goes to zero** as samples → ∞)
- Often easier and more natural when using **function approximation** (more data to train), (approx. already introduces bias)

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: First-visit Monte Carlo estimation of the state values

See '**Solving_RL.ipynb**' file in the Teams course page.

- So far, we used the grid world to build visual intuition for Monte Carlo methods.
- Now, we return to the **food truck example** for implementation.

- **Implementation notes:**
- We will use **first-visit MC**.
- Can be easily modified to **every-visit MC** by removing the check that only updates returns for the first visit of a state.
- In this example:
  - Each state is unique because the **day of the week changes after each transition**. (we cannot revisit the same state)
  - Therefore, in this example, **First-visit MC = Every-visit MC** in practice.

Implementation: First-visit Monte Carlo estimation of the state values

We start by defining a function that takes a trajectory and calculates the returns from the first visit for each state that appears in the trajectory. Works on the '**reversed**' version of teh trajectory.

$$G = r + gamma * G$$

Calculation gives:

$$G_{\tau_1} + \cdots + G_{\tau_k}$$

Why?

```python
def first_visit_return(returns, trajectory, gamma):
    G = 0
    T = len(trajectory) - 1
    for t, sar in enumerate(reversed(trajectory)):
        s, a, r = sar
        G = r + gamma * G
        first_visit = True
        for j in range(T - t):
            if s == trajectory[j][0]:
                first_visit = False
        if first_visit: # in the trajectory
            if s in returns:
                returns[s].append(G)
            else:
                returns[s] = [G]
    return returns
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: First-visit Monte Carlo estimation of the state values

Next, we implement a function that simulates the environment for a single episode with a given policy and returns the trajectory.

```
def get_trajectory(env, policy):
    trajectory = []
    observation, info = env.reset()
    state = env.state_space[observation] # convert gymnasium observation to state
    episode_over = False
    sar = [state]
    while not episode_over:
        action = choose_action(state, policy)
        action_index = foodtruck.actions_list.index(action) # action index is neec
        observation, reward, terminated, truncated, info = env.step(action_index)
        state = env.state_space[observation]
        sar.append(action)
        sar.append(reward)
        trajectory.append(sar)
        sar = [state]
        episode_over = terminated or truncated
    return trajectory
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: First-visit Monte Carlo estimation of the state values

Now, implement the first-visit return Monte Carlo function, which simulates the environment for a specified number of episodes/trajectories with a given policy. We keep track of the trajectories and average the returns for each state calculated by the first_visit_return function:

```
def first_visit_mc(env, policy, gamma, n_trajectories):
    np.random.seed(0)
    returns = {}
    v = {}
    for i in range(n_trajectories):
        trajectory = get_trajectory(env, policy)
        returns = first_visit_return(returns,
                                     trajectory,
                                     gamma)
    for s in env.state_space:
        if s in returns:
            v[s] = np.round(np.mean(returns[s]), 1)
    return v
```

Implementation: First-visit Monte Carlo estimation of the state values

Create an environment instance, Also obtain the base policy, which fills the patty inventory up to 200 or 300 with equal chance.

Then use the first-visit MC method to estimate the state values from 10,000 trajectories.

```
foodtruck = FoodTruck()
policy = base_policy(foodtruck.state_space)
```

```
v_est = first_visit_mc(foodtruck, policy, 1, 10000)
v_est
```

```
{('Mon', 0): np.float64(2515.9),
 ('Tue', 0): np.float64(1959.1),
 ('Tue', 100): np.float64(2362.2),
 ('Tue', 200): np.float64(2765.2),
 ('Wed', 0): np.float64(1411.3),
 ('Wed', 100): np.float64(1804.2),
 ('Wed', 200): np.float64(2198.9),
 ('Thu', 0): np.float64(852.9),
 ('Thu', 100): np.float64(1265.4),
 ('Thu', 200): np.float64(1644.4),
 ('Fri', 0): np.float64(301.1),
 ('Fri', 100): np.float64(696.5),
 ('Fri', 200): np.float64(1097.2)}
```

Implementation: First-visit Monte Carlo estimation of the state values

	M	Tu	W	Th	F
0	2220	1767	1240	550	433
100		1725	1350	1000	633
200			1600	1500	750
300					

$\hat{v}_\pi$ after 10 trajectories

	M	Tu	W	Th	F
0	2532	1918	1538	948	409
100		2354	1763	1203	783
200		3042	2329	3042	1042
300					

$\hat{v}_\pi$ after 100 trajectories

	M	Tu	W	Th	F
0	2518	1967	1441	870	300
100		2374	1824	1264	726
200		2803	2187	1687	1151
300					

$\hat{v}_\pi$ after 1000 trajectories

	M	Tu	W	Th	F
0	2515	1960	1405	850	295
100		2360	1805	1250	695
200		2760	2205	1650	1095
300		3205	2605	2095	1400

v_π, true values

Figure 5.5 – First-visit Monte Carlo estimates versus true state values

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: First-visit Monte Carlo estimation of the state values

Key observations:

(a) More trajectories → better estimates

As the number of sampled episodes increases, value estimates converge toward the true state values.

Increasing the trajectory count further will improve accuracy.

(b) Missing estimates for ("Tue", 200)

After 10 trajectories, this state was never visited → no value estimate.

Shows the importance of collecting **enough** trajectories.

(c) No estimates for states with 300 units of inventory

Under the **base policy**, these states are unreachable.

Therefore, we cannot estimate their value.

These states may actually be good — but the policy never explores them.

This reveals a **lack of exploration**, which we must address.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Implementation: First-visit Monte Carlo estimation of the state values

Why State Values Alone Are Not Enough?

We can now estimate **state values** using only the agent's experience — without knowing environment dynamics.

But a problem remains:
State values **alone** are not enough to **improve the policy**.

We used the state value estimates ***together with the transition probabilities*** to obtain action (q) values.

We then chose, for each state, the **action that maximized** the **q-value** of the state. Right now, since **we don't assume knowledge of the environment**, we **can't go from the state values to action values**.

Solution: Estimate the action values.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Estimating the Action-Value Function

Action-value function

$q_\pi(s, a)$:expected discounted return when
– starting in state s
– taking action a
– then following policy π

Using trajectories to estimate q_π
Trajectory:

$$s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, r_{t+2}, \ldots$$

Each observed return updates estimates of
– $q_\pi(s_t, a_t)$
–$q_\pi(s_{t+1}, a_{t+1})$,etc.

Estimating the Action-Value Function

Improving the policy

Once we have $q(s, a)$,choose

$$a^* = \arg\max_a q(s, a)$$

Main challenge:

A **deterministic policy** visits **only the actions it chooses**.

Unvisited state–action pairs → no estimates → **cannot discover** better actions.
Example: In grid world, if policy always chooses “right” at (0,0), we never try left/up/down.

This is the **exploration problem**.

Dealing with the Exploration Problem

Why exploration is needed

To evaluate *all* actions in each state, we must try them at least sometimes. Without this, we cannot learn correct action values. In general, **this is the case when we use a deterministic policy, or a stochastic policy that does not take all of the actions with some positive probability**.

Solution 1: Exploring Starts

Start each episode in a **random state** with a **random action**. Guarantees every state–action pair is tried.

Limitation: requires restarting episodes with random initialization → not practical in online/continuous interaction.

Solution 2: Soft Policies (Common Approach)

Maintain a policy that selects **every action with positive probability**:

$$\pi(a \mid s) > 0 \textit{ for all } s, a$$

Ensures continuous exploration during normal interaction. Enables discovering better actions naturally. Exploration happens *inside* the episode, not at the start.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Monte Carlo Control

- Monte Carlo control refers to methods that learn **optimal or near-optimal policies** using **sampled discounted returns**.
- In other words, it learns good policies **directly from interaction/experience**.
- Because learning depends on experience, **exploration is essential** to discover better actions.
- Next, we introduce **ε-greedy policies**, a practical form of **soft policy**, to ensure exploration during training.
- We will then examine two major variants of Monte Carlo control:
  - **On-policy methods**
  - **Off-policy methods**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Monte Carlo Control: Implementing ε-greedy policies

- As in multi-armed bandits, an **ε-greedy policy**:
  - Chooses a **random action** with probability **ε** (exploration)
  - Chooses the **best action** (argmax of the current action-value estimates) with probability **1 − ε** (exploitation)
- This ensures:
  - **All state–action pairs** continue to be explored
  - The agent still **prefers the best-known action** most of the time
- To convert a deterministic policy into an ε-greedy one:
  - Assign probability
    - **1 − ε + ε/|A(s)|** to the best action
    - **ε/|A(s)|** to every other action
  - where **|A(s)|** is the number of actions available in state *s*

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Monte Carlo Control: Implementing ε-greedy policies

```
def get_eps_greedy(actions, eps, a_best):
    prob_a = {}
    n_a = len(actions)
    for a in actions:
        if a == a_best:
            prob_a[a] = 1 - eps + eps/n_a
        else:
            prob_a[a] = eps/n_a
    return prob_a
```

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Monte Carlo Control: Implementing ε-greedy policies

- **Exploration is necessary during training to discover good actions. However, after training (during inference), we usually prefer taking the best actions rather than exploratory ones. Because of this difference, many algorithms distinguish between two policies:**
- **Behavior policy** — the policy used to generate experience during training (often exploratory).
- **Target policy** — the policy whose value we want to learn or improve (often greedy).
- **If an algorithm learns the value of the *same* policy it uses to generate data, it is on-policy. If it learns the value of a *different* policy from the one generating data, it is off-policy.**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

On-policy versus off-policy methods

Remember that the state and action values are defined **with respect to a specific policy**, hence the notation v_π and q_π.

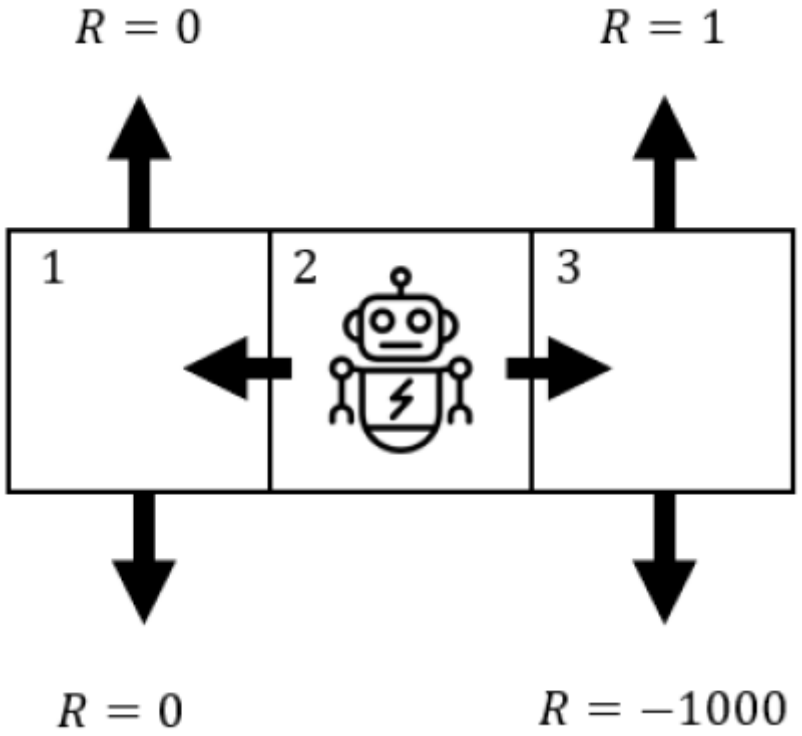
On-policy methods estimate the value functions for the *behavior policy* — the same policy that is used to generate the training data/experience. (behavior policy = target policy)

Off-policy methods estimate the value functions for a *different* policy — typically the **target policy** — while using another policy (the behavior policy) to generate experience.

This separation is useful because we often want to **decouple exploration from value estimation**, allowing us to learn optimal greedy policies while still exploring during data collection.

The impact of on-policy methods on value function estimates

- **Exploratory policies** are **usually not optimal** as they take random actions once in a while for the sake of exploration. Since on-policy methods estimate the state and action values for the behavior policy, that **sub-optimality** is **reflected in the value estimates**.
- Consider the following modified grid world example to see how involving the effects of the exploratory actions in value estimation could be potentially harmful: The robot needs to choose between going **left** or **right** in **state 2** of a 3 × 1 grid world, and between **up** and **down** in **states 1 and 3**. The **robot follows the actions perfectly**, so there is **no randomness in the environment**.



Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The impact of on-policy methods on value function estimates

- Suppose that the robot has an ***ϵ* -greedy policy**, which suggests **taking the best action with 0.99 chance**, and an **exploratory action with 0.01 chance**. (What is the ***ϵ*-value** here)?

- The **best policy** in **state 3** is to **go up with a high likelihood**. In **state 1**, the **choice does not really matter**. The state value **estimates obtained in an on-policy manner** will then be the following:

$$\mathrm{v}(1) = 0 \cdot \mathrm{p}_{\mathrm{up}} + 0 \cdot \mathrm{p}_{\mathrm{down}} = 0$$
$$\mathrm{v}(3) = 1 \cdot 0.99 - 1000 \cdot 0.01 = -9.01$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The impact of on-policy methods on value function estimates

- A policy obtained in an **on-policy fashion for state 2** will suggest **going left, towards state 1**.
- On the other hand, **in this deterministic environment**, the **robot could perfectly avoid the big penalty when there is no exploration involved**.
- An **on-policy method would fail to identify this since the exploration** influences the value estimates and yields a sub-optimal policy as a result.
- On the other hand, **in some cases**, we may want the agent to take the **impact of exploration into account** if, for example, the samples are collected using a **physical robot** and **some states are very costly to visit**.

Sample efficiency comparison between on-policy and off-policy methods

- **Sample efficiency** is critical for **deep RL** methods.
- **On-policy deep RL methods** require **discarding the past experience** once the behavior policy is updated.
  - because old trajectories reflect the previous policy and no longer provide accurate estimates for the new policy.
  - for example, if a robot learns with $\varepsilon = 0.1$ and then the policy is updated to $\varepsilon = 0.05$, the old $\varepsilon = 0.1$ trajectories cannot be used to evaluate the new policy accurately.
- **Off-policy deep RL methods, however, can reuse the past experience again and again**. This is a significant advantage in sample efficiency, especially if it is costly to obtain the experience.

Sample efficiency comparison between on-policy and off-policy methods

- **Off-policy** methods' **ability to use the experience generated by a policy other than the behavior policy** comes in handy is when we want to **warm-start** the training based on the data generated by a non-RL controller, such as classical PID controllers or a human operator.
- **Warm-start:** using existing data or experience to **initialize learning**, so the agent starts closer to good behavior and requires fewer new samples.
- This is especially useful when the environment is hard/expensive to simulate or collect experience from.
- **Summary**
  - **On-policy:** cannot reliably use past data from other policies → warm-start is not practical.
  - **Off-policy:** can use past data from any policy → warm-start works well.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Advantages of on-policy methods

The natural question is then **why do we even talk about on-policy methods instead of just ignoring them**? There are **several advantages of on-policy methods**:

- **If** the **cost of sampling** from the environment is **high** (the actual environment rather than a simulation), we may want to have a policy that **reflects the impact of exploration** to **avoid catastrophic outcomes**. (In **on-policy** methods, Q-values **already reflect** the expected returns under the **current exploratory policy**, so selecting the highest Q-value naturally accounts for the policy's exploration.) (**In off-policy** learning, Q-values correspond to the **target policy** and do **not directly reflect** the exploratory outcomes of the behavior policy.)
- **Off-policy** methods, when combined **with function approximators**, could **have issues with converging to a good policy**. We will discuss later.
- **On-policy** methods are **easier to work with when the action space is continuous**, again, as we will discuss later.